there forever in any sequence that starts with $r.q$ (as explained above, it is removed only after the sequence $r.r.s.s.s$). This corresponds to the left chunk of "SSB traces" in Fig. 3.

Optimal-DPOR solves this problem by replacing the backtrack set with a structure called a *wakeup tree*. This tree contains initial fragments of executions that are guaranteed not to encounter sleep set blocking. In the example, the Optimal-DPOR algorithm will handle the race between `(1)` and `(7)` by adding the sequence $r.r.s.s.s$ to the wakeup tree. The point is that after $r.r.s.s.s$, the process p has been removed from the sleep set, and so sleep set blocking is avoided.

Events Versus Transitions. A majority of approaches to partial order reduction define dependencies as a relation between syntactic statements in the program [Clarke et al. 1999; Flanagan and Godefroid 2005b; Godefroid 1996, 2005; Lauterburg et al. 2010; Tasharofi et al. 2012; Valmari 1991], often referred to as *transitions*. The notion of transition is sometimes refined by including the local state from which it is executed. There are also also approaches where dependencies are defined between occurrences of transitions in a given execution [Lei and Carver 2006; Sen and Agha 2007], sometimes referred to as *events*. In general, events allow finer distinctions when defining dependencies than transitions. For instance, two send operations need not be dependent unless the order in which the two messages are received is significant. This information may not be available if the send operations are regarded as syntactic statements, even if the local states of the sending process is considered. On the other hand, if the send operations are considered in the context of a particular execution, then the definition of dependencies can be appropriately refined. Thus, a reduction technique in which the definition of dependencies is based on events can achieve better reduction than one based on transitions, for instance when applied to programs that employ message passing.

We illustrate that events can be more suitable than transitions on a small example.

```
p :                          q :
send(q, first);              receive m1;
send(q, second);             receive m2;
                             L := append(append(L, m1), m2);
```

Here, process p sends two messages to process q, and process q receives two messages and appends them to the list `L`. Messages are received in FIFO order. For this example, we assume that `receive` statements must return a message but are non-blocking; if a `receive` is ever attempted by a process when its incoming message queue is empty, the process dies.

For this example, there are three possible outcomes of executing the program, characterized by the number of messages received by q. Thus, we would want to define a definition of dependencies, which gives rise to three different Mazurkiewicz traces: $p.p.q.q$, $p.q.q.p$ and $q.p.p$. We note that a transition-based definition, which regards any `send` statement as dependent with any `receive` statement will give rise to four Mazurkiewicz traces. To reduce this number, we must define the transmission of the second message by p to be independent of the reception of the first message by q, thereby making the executions $p.p.q.q$ and $p.q.p.q$ equivalent. This is possible if the definition of dependencies is based on events, since that allows to take the context of the execution into account. In this particular example, this difference allows us to reduce the number of explored transition sequences from four to three. In larger programs, where message queues may contain several messages, the reduction in explored interleavings can be substantial.

3 FRAMEWORK

In this section, we introduce the technical background material. First, we present the general model of concurrent systems for which the algorithms are formulated, thereafter the assumptions on the happens-before relation, and finally the notions of independence and races.